

Assignment 3

Monte Carlo Methods for Financial Applications

5MA178

Markus Ådahl

Sam Moeini and André Johansson

samo0171, anyp0002

June 9, 2024

Contents

1	Task 1: Dynamic hedgin of a European call option	3
1.1	Method for Task 1	3
1.2	Results: Task 1	3
1.3	Discussion: Task 1	4
2	Task 2: Delta and vega for an Asian call	5
2.1	Method for Task 2 a	5
2.2	Results: Task 2a	6
2.3	Diskussion: Task 2a	6
3	Task 4: Implied volatilities and greeks for American options	7
3.1	Method for Task 4	7
3.2	Results: Task 4	8
3.3	Discussion: Task 4	9
4	Python Code to solve all tasks	10

1 Task 1: Dynamic hedging of a European call option

1.1 Method for Task 1

1. Calculate the call option price using the Black-Scholes formula
2. For each rebalancing frequency m , simulate n stock price paths using geometric Brownian where Z is a standard normal random variable.
3. For each time step, calculate the delta and the option price using the Black-Scholes formula.
4. At each time step calculate the hedging portfolio value using the delta and the option price. Calculate the present value of the profit or loss from the hedging portfolio.
5. For each rebalancing frequency calculate the mean and standard deviation of the present values from the simulations. Plot histograms of the present values for each strategy on the same plot.

1.2 Results: Task 1

Table 1: Comparison of Call Option Pricing Methods

Strategy	Mean	Standard deviation
Daily	0.0027	0.5610
Weekly	0.0058	1.2280
Monthly	0.01902	2.4069

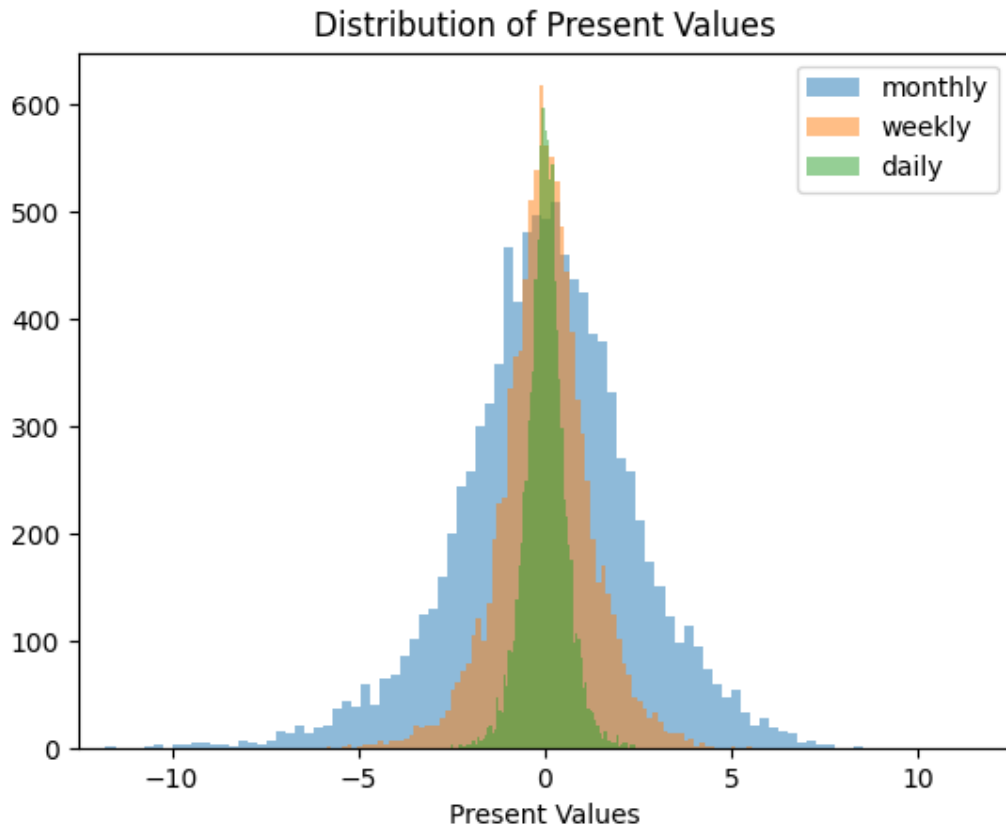


Figure 1: Plot for Task 1 b, distribution of the present values for each strategy

1.3 Discussion: Task 1

What effect does the rebalancing frequency have?

More frequent rebalancing provides better hedging accuracy and risk management but involves higher transaction costs. Less frequent rebalancing reduces transaction costs but increases the risk of larger hedging errors and potential losses. The optimal frequency balances these factors based on specific needs.

2 Task 2: Delta and vega for an Asian call

2.1 Method for Task 2 a

1. Create a function to simulate stock paths for an Asian option using geometric Brownian motion. Generate standard normal random variables. Initialize the stock price array and simulate the stock prices over time.
2. Define a function to calculate the payoff of an Asian call option based on the average stock price. Calculate the discounted payoff to the present value.
3. Create a function to calculate vega using forward differences by simulating stock paths with increased volatility and calculating the payoff. Create a function to calculate vega using central differences by simulating stock paths with both increased and decreased volatility, and calculating the payoff. Create a function to calculate delta using forward differences by simulating stock paths with increased initial stock price and calculating the payoff. Create a function to calculate delta using central differences by simulating stock paths with both increased and decreased initial stock prices, and calculating the payoff.
4. Iterate over each combination of rebalancing frequency and strike price. For each combination call the functions to calculate vega (FD and CD) and delta (FD and CD).

2.2 Results: Task 2a

Greek & method	K = 80 (m=12)	K = 80 (m=52)	K = 100 (m=12)	K = 100 (m=52)	K = 120 (m=12)	K = 120 (m=52)
Delta (f.d.)	0.9668	0.9722	0.5851	0.5797	0.1073	0.0937
Delta (c.d.)	0.9649	0.9693	0.5617	0.5550	0.0992	0.08525
Delta (p.d.)						
Vega (f.d.)	3.7254	3.0029	23.2472	22.7156	11.5786	9.9921
Vega (c.d.)	3.1358	1.7494	23.8426	22.7495	10.7188	9.5869
Vega (p.d.)						

Table 2: Greeks calculated using different methods for various strike prices and rebalancing frequencies.

2.3 Diskussion: Task 2a

Discussion: Are these vegas higher/lower than regular call vegas? Why is that?

Vegas for Asian options are generally lower than those for regular European options. This is because Asian options' payoffs are based on the average price of the underlying asset over a period. Therefore the value of Asian options is less sensitive to changes in volatility resulting in lower vegas.

3 Task 4: Implied volatilities and greeks for American options

3.1 Method for Task 4

1. Define variables.
2. Load and parse data from CSV files.
3. Calculate the time step and the number of steps.
4. Identify specific strikes and compute the strangle price.
5. Define binomial tree pricing and implied volatility functions.
6. Extract relevant strike prices and prices.
7. Calculate implied volatilities for each strike.
8. Plot the implied volatilities.
9. Compute option greeks (Delta, Theta, Gamma).

3.2 Results: Task 4

The result for task 4 a was 23,4.

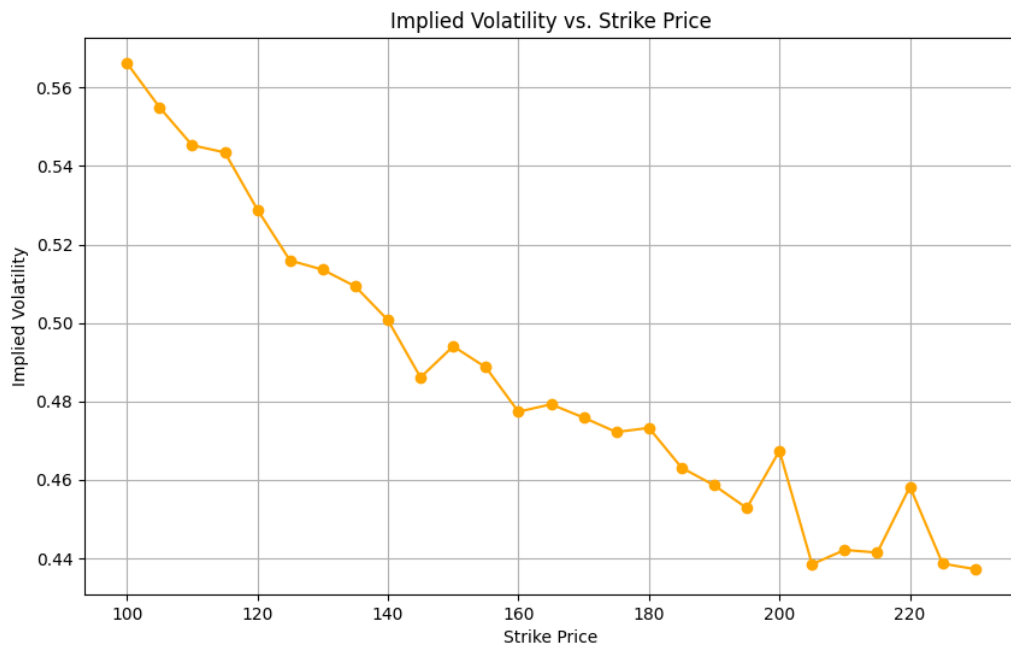


Figure 2: Plot for Task 4 b, estimated implied volatilities.

Strike Price	Implied Volatility (%)
100.0	56.62%
105.0	55.49%
110.0	54.53%
115.0	54.34%
120.0	52.89%
125.0	51.59%
130.0	51.36%
135.0	50.93%
140.0	50.07%
145.0	48.61%
150.0	49.41%
155.0	48.87%
160.0	47.74%
165.0	47.93%
170.0	47.59%
175.0	47.22%
180.0	47.33%
185.0	46.31%
190.0	45.86%
195.0	45.28%
200.0	46.74%
205.0	43.85%
210.0	44.22%
215.0	44.15%
220.0	45.83%
225.0	43.87%
230.0	43.73%

Table 3: Strike Prices and their respective Implied Volatility

Option Greek	Value
Delta	-0.408
Theta	-0.075
Gamma	0.007

Table 4: Option Greeks

3.3 Discussion: Task 4

The shape of the plot from task 4 b is a skew. This can be observed both visually in the graph or by inspecting the numbers from the table where we can see a downward trend in the implied volatility as the strike prices get increasingly higher.

4 Python Code to solve all tasks

```
import numpy as np
import pandas as pd
from scipy.stats import norm, qmc
import matplotlib.pyplot as plt
from scipy.stats.mstats import gmean
from scipy.stats import norm, lognorm

"""
#Task 1

s0 = 100
K = 90
r = 0.03
sigma = 0.3
T = 1
n = 10000
m = [252, 52, 12]

def black_scholes(s0, K, r, T, sigma):
    d1 = (np.log(s0 / K) + (r + 0.5 * sigma ** 2) * T) / (sigma
        * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    price = s0 * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2
        )
    return price

def delta_hedging(s0, K, r, sigma, T, n, Z, m):
    dt = T / m
    S = np.zeros((n, m + 1))
    S[:, 0] = s0
    for i in range(1, m + 1):
        S[:, i] = S[:, i - 1] * np.exp((r - 0.5 * sigma ** 2) *
            dt + sigma * np.sqrt(dt) * Z[:, i - 1])

    delta = np.zeros((n, m))
    call = np.zeros((n, m))
    for i in range(1, m + 1):
        t = i * dt
        d1 = (np.log(S[:, i - 1] / K) + (r + 0.5 * sigma ** 2) *
            (T - t)) / (sigma * np.sqrt(T - t))
        delta[:, i - 1] = norm.cdf(d1)
        call[:, i - 1] = black_scholes(S[:, i - 1], K, r, T - t,
            sigma)

    y = np.zeros((n, m))
```

```

portfolio = np.zeros((n, m + 1))
portfolio[:, 0] = call[:, 0]
PresentValue = np.zeros((n, m))

for i in range(1, m + 1):
    y[:, i - 1] = call[:, i - 1] - delta[:, i - 1] * S[:, i - 1]
    portfolio[:, i] = delta[:, i - 1] * S[:, i] + np.exp(r * dt) * y[:, i - 1]
    PresentValue[:, i - 1] = np.exp(-r * i * dt) * (
        portfolio[:, i - 1] - call[:, i - 1])

PresentValue_sum = np.sum(PresentValue, axis=1)
CF_mean = np.mean(PresentValue_sum)
CF_std = np.std(PresentValue_sum)
return CF_mean, CF_std, PresentValue_sum

results = {}
for t in m:
    Z = np.random.standard_normal((n, t + 1))
    mean, std, PresentValue_sum = delta_hedging(s0, K, r, sigma,
        T, n, Z, t)
    results[t] = PresentValue_sum
    print(f"Mean for m = {t} is {mean}")
    print(f"Std for m = {t} is {std}")

plt.hist(results[12], bins=100, alpha=0.5, label='monthly')
plt.hist(results[52], bins=100, alpha=0.5, label='weekly')
plt.hist(results[252], bins=100, alpha=0.5, label='daily')
plt.xlabel('Present Values')
plt.title('Distribution of Present Values')
plt.xlim(-12.5, 12.5)
plt.legend()
plt.show()

"""

"""

# 2 a)

risk_free_rate = 0.03
volatility = 0.2
initial_stock_price = 100
time_to_maturity = 1
strike_prices = [80, 100, 120]
rebalancing_frequencies = [12, 52]

```

```

n = 20000
h_delta = 1.0
h_vega = 0.01

def simulate_stock_paths(S0, r, sigma, T, m, Z=None):
    dt = T / m
    if Z is None:
        Z = np.random.standard_normal((n, m))
    S = np.zeros((n, m + 1))
    S[:, 0] = S0
    for t in range(1, m + 1):
        S[:, t] = S[:, t - 1] * np.exp((r - 0.5 * sigma ** 2) *
            dt + sigma * np.sqrt(dt) * Z[:, t - 1])
    return S, Z

def asian_call_payoff(S, K, r, T):
    S_mean = np.mean(S[:, 1:], axis=1)
    payoff = np.maximum(S_mean - K, 0)
    discounted_payoff = np.mean(payoff) * np.exp(-r * T)
    return discounted_payoff

def calculate_vega_FD(T, sigma, r, m, K, h_vega, S0):
    S, Z = simulate_stock_paths(S0, r, sigma, T, m)
    V_h = asian_call_payoff(S, K, r, T)
    S_plus, _ = simulate_stock_paths(S0, r, sigma + h_vega, T, m,
        Z)
    V_plus = asian_call_payoff(S_plus, K, r, T)
    vega_fd = (V_plus - V_h) / h_vega
    return vega_fd

def calculate_vega_CD(T, sigma, r, m, K, h_vega, S0):
    S, Z = simulate_stock_paths(S0, r, sigma, T, m)
    S_plus, _ = simulate_stock_paths(S0, r, sigma + h_vega, T, m,
        Z)
    S_minus, _ = simulate_stock_paths(S0, r, sigma - h_vega, T,
        m, Z)
    V_plus = asian_call_payoff(S_plus, K, r, T)
    V_minus = asian_call_payoff(S_minus, K, r, T)
    vega_cd = (V_plus - V_minus) / (2 * h_vega)
    return vega_cd

def calculate_delta_FD(T, sigma, r, m, K, h_delta, S0):
    S, Z = simulate_stock_paths(S0, r, sigma, T, m)
    V_h = asian_call_payoff(S, K, r, T)
    S_plus, _ = simulate_stock_paths(S0 + h_delta, r, sigma, T,
        m, Z)
    V_plus = asian_call_payoff(S_plus, K, r, T)
    delta_fd = (V_plus - V_h) / h_delta

```

```

    return delta_fd

def calculate_delta_CD(T, sigma, r, m, K, h_delta, S0):
    S, Z = simulate_stock_paths(S0, r, sigma, T, m)
    S_plus, _ = simulate_stock_paths(S0 + h_delta, r, sigma, T,
                                     m, Z)
    S_minus, _ = simulate_stock_paths(S0 - h_delta, r, sigma, T,
                                     m, Z)
    V_plus = asian_call_payoff(S_plus, K, r, T)
    V_minus = asian_call_payoff(S_minus, K, r, T)
    delta_cd = (V_plus - V_minus) / (2 * h_delta)
    return delta_cd

for rebalancing_freq in rebalancing_frequencies:
    for strike_price in strike_prices:
        vega_fd_value = calculate_vega_FD(time_to_maturity,
                                          volatility, risk_free_rate, rebalancing_freq,
                                          strike_price, h_vega, initial_stock_price)
        vega_cd_value = calculate_vega_CD(time_to_maturity,
                                          volatility, risk_free_rate, rebalancing_freq,
                                          strike_price, h_vega, initial_stock_price)
        delta_fd_value = calculate_delta_FD(time_to_maturity,
                                             volatility, risk_free_rate, rebalancing_freq,
                                             strike_price, h_delta, initial_stock_price)
        delta_cd_value = calculate_delta_CD(time_to_maturity,
                                             volatility, risk_free_rate, rebalancing_freq,
                                             strike_price, h_delta, initial_stock_price)

        print(f"For rebalancing frequency = {rebalancing_freq}
              and strike price = {strike_price}:")
        print(f"  Vega (FD): {vega_fd_value}")
        print(f"  Vega (CD): {vega_cd_value}")
        print(f"  Delta (FD): {delta_fd_value}")
        print(f"  Delta (CD): {delta_cd_value}")

"""

"""

#Task 4

puts_filename = 'TSLA_puts.csv'
calls_filename = 'TSLA_calls.csv'
put_data = []
call_data = []
S0 = 165.8
r = 0.05

```

```

T = (np.datetime64('2024-09-20') - np.datetime64('2024-03-15')).
    astype(float) / 365.25
N = round(T * 52)
# Create lists to save different values 7
# for both the put and call list

with open(puts_filename, 'r') as file:
    reader = csv.reader(file, delimiter=';')

    # Skip the header line
    next(reader)

    # Read the rest of the lines and store them in the data list
    for row in reader:
        put_data.append(row)

# Convert the list of lists to a NumPy array
put_values = np.array(put_data)
contract_names_puts = put_values[:, 0]
puts = put_values[:, 1:5].astype(float)

with open(calls_filename, 'r') as file:
    reader = csv.reader(file, delimiter=';')

    # Skip the header line
    next(reader)

    # Read the rest of the lines and store them in the data list
    for row in reader:
        call_data.append(row)

# Convert the list of lists to a NumPy array
call_values = np.array(call_data)
contract_names_calls = call_values[:, 0]
calls = call_values[:, 1:5].astype(float)

# a

for i in range(puts.shape[0]):
    strike_put = puts[i,0]
    strike_call = calls[i,0]

    if strike_put == 135:
        put_row = i
    if strike_call == 195:
        call_row = i

```

```

strangle_price = puts[put_row,-1] + calls[call_row,-1]
print("The price for the long strangle with strikes 135 and 195
      is: ",strangle_price)

# b

def binomial_tree_put_pricing(S, K, r, T, volatility, N):
    dt = 1 / 52
    u = np.exp(volatility * np.sqrt(dt))
    d = 1 / u
    p = (np.exp(r * dt) - d) / (u - d)
    q = 1 - p
    discount_factor = np.exp(-r * dt)

    # Initialize stock prices and option values
    stock_prices = np.zeros((N + 1, N + 1))
    exercise_value = np.zeros((N + 1, N + 1))

    # Set up stock prices at maturity
    for i in range(N + 1):
        stock_prices[i, N] = S * (u ** (N - i)) * (d ** i)
        exercise_value[i, N] = max(K - stock_prices[i, N], 0)

    # Backward induction
    for j in range(N - 1, -1, -1):
        for i in range(j + 1):
            stock_prices[i, j] = S * (u ** (j - i)) * (d ** i)
            exercise_value[i, j] = max(K - stock_prices[i, j],
                                         0)
            continuation_value = discount_factor * (p *
                exercise_value[i, j + 1] + q * exercise_value[i +
                    1, j + 1])
            exercise_value[i, j] = max(exercise_value[i, j],
                                         continuation_value)

    return exercise_value[0, 0]
def implied_volatility(S,K,r,T,market_price,N):
    low = 0
    high = 1
    mid = (low+high)/2
    error = 0.0001
    price = binomial_tree_put_pricing(S,K,r,T,mid,N)

    while abs(price-market_price)>error:
        mid = (low+high)/2
        price = binomial_tree_put_pricing(S,K,r,T,mid,N)
        if abs(price-market_price) < error:
            return mid

```

```

        elif price < market_price:
            low = mid
        else:
            high = mid

    return mid

relevant_list = []

for i in range(puts.shape[0]):
    strike = puts[i,0]
    if strike >= 100 and strike <= 230:
        price = (puts[i,2] + puts[i,3]) / 2
        # relevant_list = puts[i,0]
        relevant_list.append([strike, price])

relevant_array = np.array(relevant_list)

implied_volatilities = []

for i in range(relevant_array.shape[0]):
    implied_vol = implied_volatility(S0, relevant_array[i, 0], r
        , T, relevant_array[i, 1], N)
    implied_volatilities.append(implied_vol)

implied_volatilities = np.array(implied_volatilities)

# Plotting the implied volatilities
plt.figure(figsize=(10, 6))
plt.plot(relevant_array[:, 0], implied_volatilities, linestyle
        ='--', marker = 'o', color='orange')
plt.xlabel('Strike Price')
plt.ylabel('Implied Volatility')
plt.title('Implied Volatility vs. Strike Price')
plt.grid(True)
plt.show()

implied_vol_table = pd.DataFrame({
    'Strike Price': relevant_array[:, 0],
    'Implied Volatility': implied_volatilities
})
print(implied_vol_table)

# c

```



```

def calculate_greeks(S,K,r,T,volatility,N):
    dt = 1/52
    u = np.exp(volatility * np.sqrt(dt))
    d = 1/u
    p = (np.exp(r*dt)-d)/(u-d)
    q = 1-p
    discount_factor = np.exp(-r*dt)
    stock_prices = np.ones((N,N))
    exercise_value = np.zeros((N,N))
    continuation_value = np.zeros((N,N))

    for rows in range(N):
        for cols in range(rows,N,1):
            stock_prices[rows,cols] = S * u**cols * d**(rows*2)
            exercise_value[rows,cols] = np.maximum(K-
                stock_prices[rows,cols],0)
    option_value = np.copy(exercise_value)
    continuation_value[:, -1] = 0

    for rows in range(N-1):
        for cols in range(N-1):
            continuation_value[N-rows-2,N-cols-2] =
                discount_factor * (option_value[N-rows-2,N-cols-1]*p +
                    option_value[N-rows-1,N-cols-1]*q)
            option_value[N-rows-2,N-cols-2] = np.maximum(
                continuation_value[N-rows-2,N-cols-2],
                exercise_value[N-rows-2,N-cols-2])

    delta = (option_value[0,1]-option_value[1,1]) / (
        stock_prices[0,1]-stock_prices[1,1])
    theta = ((option_value[1,2] - option_value[0,0]) / (2*dt
        *252))
    Sg1 = ((option_value[0,2]-option_value[1,2]) / (stock_prices
        [0,2]-stock_prices[0,0]))
    Sg2 = ((option_value[1,2]-option_value[2,2]) / (stock_prices
        [0,0]-stock_prices[2,2]))
    gamma = (Sg1 - Sg2) / ((stock_prices[0,2] - stock_prices
        [2,2])/2)
    return delta , theta , gamma

for i in range(relevant_array.shape[0]):
    if relevant_array[i,0] == 165:
        current_volatility = implied_volatilities[i]
        break

delta , theta , gamma = calculate_greeks(S0,165,r,T,
    current_volatility,N)
print("Delta is: ", delta)

```

```
print("Theta is: ", theta)
print("Gamma is: ", gamma)

"""
```